Death-Note-Zap-Report

Sites: https://deathnote.vuln http://deathnote.vuln

Generated on Sun, 18 Oct 2026 12:39:51

ZAP Version: 2.17.0

ZAP by Checkmarx

Summary of Alerts

Risk Level	Number of Alerts
High	1
Medium	5
Low	4
Informational	4

Insights

Level	Reason	Site	Description	Statistic
Info	Informational		Percentage of network failures	1 %
Info	Informational	http://deathnote.vuln	Percentage of responses with status code 2xx	56 %
Info	Informational	http://deathnote.vuln	Percentage of responses with status code 3xx	15 %
Info	Exceeded Low	http://deathnote.vuln	Percentage of responses with status code 4xx	27 %
Info	Informational	http://deathnote.vuln	Percentage of endpoints with content type application/javascript	10 %
Info	Informational	http://deathnote.vuln	Percentage of endpoints with content	12 %

			type application /json	
Info	Informational	http://deathnote.vuln	Percentage of endpoints with content type application /rss+xml	10 %
Info	Informational	http://deathnote.vuln	Percentage of endpoints with content type application /xml	2 %
Info	Informational	http://deathnote.vuln	Percentage of endpoints with content type image /jpeg	12 %
Info	Informational	http://deathnote.vuln	Percentage of endpoints with content type text /css	8 %
Info	Informational	http://deathnote.vuln	Percentage of endpoints with content type text /html	35 %
Info	Informational	http://deathnote.vuln	Percentage of endpoints with content type text /plain	4 %
Info	Informational	http://deathnote.vuln	Percentage of endpoints with content type text /xml	4 %
Info	Informational	http://deathnote.vuln	Percentage of endpoints with method GET	97 %
			Percentage of endpoints	

Info	Informational	http://deathnote.vuln	with method POST	2 %
Info	Informational	http://deathnote.vuln	Count of total endpoints	48
Info	Exceeded Low	http://deathnote.vuln	Percentage of slow responses	22 %
Info	Informational	https://deathnote.vuln	Percentage of endpoints with method GET	100 %
Info	Informational	https://deathnote.vuln	Count of total endpoints	1

Alerts

Name	Risk Level	Number of Instances
SQL Injection	High	1
Absence of Anti-CSRF Tokens	Medium	2
Content Security Policy (CSP) Header Not Set	Medium	Systemic
Directory Browsing	Medium	Systemic
HTTP Only Site	Medium	1
Missing Anti-clickjacking Header	Medium	Systemic
In Page Banner Information Leak	Low	1
Private IP Disclosure	Low	7
Server Leaks Version Information via "Server" HTTP Response Header Field	Low	Systemic
X-Content-Type-Options Header Missing	Low	Systemic
Charset Mismatch	Informational	1
Information Disclosure - Suspicious Comments	Informational	1
User Agent Fuzzer	Informational	Systemic
User Controllable HTML Element Attribute (Potential XSS)	Informational	1

Alert Detail

High	SQL Injection
Description	SQL injection may be possible.
URL	http://deathnote.vuln/wordpress/wp-comments-post.php
Node Name	http://deathnote.vuln/wordpress/wp-comments-post.php ()(author,comment, comment_parent,comment_post_ID,email,submit,url,wp-comment-cookies-consent)
Method	POST
Attack	3-2

Evidence	
Other Info	The original page results were successfully replicated using the expression [3-2] as the parameter value The parameter value being modified was stripped from the HTML output for the purposes of the comparison.
Instances	1
Solution	Do not trust client side input, even if there is client side validation in place. In general, type check all data on the server side. If the application uses JDBC, use PreparedStatement or CallableStatement, with parameters passed by '?' If the application uses ASP, use ADO Command Objects with strong type checking and parameterized queries. If database Stored Procedures can be used, use them. Do *not* concatenate strings into queries in the stored procedure, or use 'exec', 'exec immediate', or equivalent functionality! Do not create dynamic SQL queries using simple string concatenation. Escape all data received from the client. Apply an 'allow list' of allowed characters, or a 'deny list' of disallowed characters in user input. Apply the principle of least privilege by using the least privileged database user possible. In particular, avoid using the 'sa' or 'db-owner' database users. This does not eliminate SQL injection, but minimizes its impact. Grant the minimum database access that is necessary for the application.
Reference	https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html
CWE Id	89
WASC Id	19
Plugin Id	40018

Medium	**Absence of Anti-CSRF Tokens**
Description	No Anti-CSRF tokens were found in a HTML submission form. A cross-site request forgery is an attack that involves forcing a victim to send an HTTP request to a target destination without their knowledge or intent in order to perform an action as the victim. The underlying cause is application functionality using predictable URL /form actions in a repeatable way. The nature of the attack is that CSRF exploits the trust that a web site has for a user. By contrast, cross-site scripting (XSS) exploits the trust that a user has for a web site. Like XSS, CSRF attacks are not necessarily cross-site, but they can be. Cross-site request forgery is also known as CSRF, XSRF, one-click attack, session riding, confused deputy, and sea surf. CSRF attacks are effective in a number of situations, including: * The victim has an active session on the target site. * The victim is authenticated via HTTP auth on the target site. * The victim is on the same local network as the target site. CSRF has primarily been used to perform an action against a target site using the victim's privileges, but recent techniques have been discovered to disclose information by gaining

	access to the response. The risk of information disclosure is dramatically increased when the target site is vulnerable to XSS, because XSS can be used as a platform for CSRF, allowing the attack to operate within the bounds of the same-origin policy.
URL	http://deathnote.vuln/wordpress/index.php/2021/07/19/kira/
Node Name	http://deathnote.vuln/wordpress/index.php/2021/07/19/kira/
Method	GET
Attack	
Evidence	<form action="http://deathnote.vuln/wordpress/wp-comments-post.php" method="post" id="commentform" class="comment-form" novalidate>
Other Info	No known Anti-CSRF token [anticsrf, CSRFToken, __RequestVerificationToken, csrfmiddlewaretoken, authenticity_token, OWASP_CSRFTOKEN, anoncsrf, csrf_token, _csrf, _csrfSecret, __csrf_magic, CSRF, _token, _csrf_token, _csrfToken] was found in the following HTML form: [Form 1: "author" "comment_parent" "comment_post_ID" "email" "submit" "url" "wp-comment-cookies-consent"].
URL	http://deathnote.vuln/wordpress/index.php/2021/07/19/kira/?replytocom=1
Node Name	http://deathnote.vuln/wordpress/index.php/2021/07/19/kira/ (replytocom)
Method	GET
Attack	
Evidence	<form action="http://deathnote.vuln/wordpress/wp-comments-post.php" method="post" id="commentform" class="comment-form" novalidate>
Other Info	No known Anti-CSRF token [anticsrf, CSRFToken, __RequestVerificationToken, csrfmiddlewaretoken, authenticity_token, OWASP_CSRFTOKEN, anoncsrf, csrf_token, _csrf, _csrfSecret, __csrf_magic, CSRF, _token, _csrf_token, _csrfToken] was found in the following HTML form: [Form 1: "author" "comment_parent" "comment_post_ID" "email" "submit" "url" "wp-comment-cookies-consent"].
Instances	2
Solution	Phase: Architecture and Design Use a vetted library or framework that does not allow this weakness to occur or provides constructs that make this weakness easier to avoid. For example, use anti-CSRF packages such as the OWASP CSRFGuard. Phase: Implementation Ensure that your application is free of cross-site scripting issues, because most CSRF defenses can be bypassed using attacker-controlled script. Phase: Architecture and Design Generate a unique nonce for each form, place the nonce into the form, and verify the nonce upon receipt of the form. Be sure that the nonce is not predictable (CWE-330). Note that this can be bypassed using XSS. Identify especially dangerous operations. When the user performs a dangerous operation, send a separate confirmation request to ensure that the user intended to perform that operation. Note that this can be bypassed using XSS. Use the ESAPI Session Management control. This control includes a component for CSRF. Do not use the GET method for any request that triggers a state change.

	Phase: Implementation Check the HTTP Referer header to see if the request originated from an expected page. This could break legitimate functionality, because users or proxies may have disabled sending the Referer for privacy reasons.
Reference	https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html https://cwe.mitre.org/data/definitions/352.html
CWE Id	352
WASC Id	9
Plugin Id	10202

Medium	**Content Security Policy (CSP) Header Not Set**
Description	Content Security Policy (CSP) is an added layer of security that helps to detect and mitigate certain types of attacks, including Cross Site Scripting (XSS) and data injection attacks. These attacks are used for everything from data theft to site defacement or distribution of malware. CSP provides a set of standard HTTP headers that allow website owners to declare approved sources of content that browsers should be allowed to load on that page — covered types are JavaScript, CSS, HTML frames, fonts, images and embeddable objects such as Java applets, ActiveX, audio and video files.
URL	http://deathnote.vuln
Node Name	http://deathnote.vuln
Method	GET
Attack	
Evidence	
Other Info	
URL	http://deathnote.vuln/sitemap.xml
Node Name	http://deathnote.vuln/sitemap.xml
Method	GET
Attack	
Evidence	
Other Info	
URL	http://deathnote.vuln/wordpress/
Node Name	http://deathnote.vuln/wordpress/
Method	GET
Attack	
Evidence	
Other Info	
URL	http://deathnote.vuln/wordpress/index.php/2021/07/19/kira/
Node Name	http://deathnote.vuln/wordpress/index.php/2021/07/19/kira/
Method	GET
Attack	

Evidence	
Other Info	
URL	http://deathnote.vuln/wordpress/index.php/hint/
Node Name	http://deathnote.vuln/wordpress/index.php/hint/
Method	GET
Attack	
Evidence	
Other Info	
Instances	Systemic
Solution	Ensure that your web server, application server, load balancer, etc. is configured to set the Content-Security-Policy header.
Reference	https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CSP https://cheatsheetseries.owasp.org/cheatsheets/Content_Security_Policy_Cheat_Sheet.html https://www.w3.org/TR/CSP/ https://w3c.github.io/webappsec-csp/ https://web.dev/articles/csp https://caniuse.com/#feat=contentsecuritypolicy https://content-security-policy.com/
CWE Id	693
WASC Id	15
Plugin Id	10038

Medium	**Directory Browsing**
Description	It is possible to view the directory listing. Directory listing may reveal hidden scripts, include files, backup source files, etc. which can be accessed to read sensitive information.
URL	http://deathnote.vuln/wordpress/wp-content/themes/twentytwentyone/assets/
Node Name	http://deathnote.vuln/wordpress/wp-content/themes/twentytwentyone/assets/
Method	GET
Attack	http://deathnote.vuln/wordpress/wp-content/themes/twentytwentyone/assets/
Evidence	Parent Directory
Other Info	
URL	http://deathnote.vuln/wordpress/wp-content/themes/twentytwentyone/assets/css/
Node Name	http://deathnote.vuln/wordpress/wp-content/themes/twentytwentyone/assets/css/
Method	GET
Attack	http://deathnote.vuln/wordpress/wp-content/themes/twentytwentyone/assets/css/
Evidence	Parent Directory
Other Info	
URL	http://deathnote.vuln/wordpress/wp-content/themes/twentytwentyone/assets/js/
Node Name	http://deathnote.vuln/wordpress/wp-content/themes/twentytwentyone/assets/js/
Method	GET

Attack	http://deathnote.vuln/wordpress/wp-content/themes/twentytwentyone/assets/js/
Evidence	Parent Directory
Other Info	
URL	http://deathnote.vuln/wordpress/wp-content/uploads/
Node Name	http://deathnote.vuln/wordpress/wp-content/uploads/
Method	GET
Attack	http://deathnote.vuln/wordpress/wp-content/uploads/
Evidence	Parent Directory
Other Info	
URL	http://deathnote.vuln/wordpress/wp-content/uploads/2021/07/
Node Name	http://deathnote.vuln/wordpress/wp-content/uploads/2021/07/
Method	GET
Attack	http://deathnote.vuln/wordpress/wp-content/uploads/2021/07/
Evidence	Parent Directory
Other Info	
Instances	Systemic
Solution	Disable directory browsing. If this is required, make sure the listed files does not induce risks.
Reference	https://httpd.apache.org/docs/current/mod/core.html#options
CWE Id	548
WASC Id	48
Plugin Id	0

Medium	**HTTP Only Site**
Description	The site is only served under HTTP and not HTTPS.
URL	http://deathnote.vuln/wordpress/index.php/wp-json/
Node Name	https://deathnote.vuln/wordpress/index.php/wp-json/
Method	GET
Attack	
Evidence	
Other Info	Failed to connect. ZAP attempted to connect via: https://deathnote.vuln/wordpress/index.php/wp-json/
Instances	1
Solution	Configure your web or application server to use SSL (https).
Reference	https://cheatsheetseries.owasp.org/cheatsheets/Transport_Layer_Protection_Cheat_Sheet.html https://letsencrypt.org/
CWE Id	311
WASC Id	4
Plugin Id	10106

Medium	Missing Anti-clickjacking Header
Description	The response does not protect against 'ClickJacking' attacks. It should include either Content-Security-Policy with 'frame-ancestors' directive or X-Frame-Options.
URL	http://deathnote.vuln
Node Name	http://deathnote.vuln
Method	GET
Attack	
Evidence	
Other Info	
URL	http://deathnote.vuln/wordpress/
Node Name	http://deathnote.vuln/wordpress/
Method	GET
Attack	
Evidence	
Other Info	
URL	http://deathnote.vuln/wordpress/index.php/2021/07/19/kira/
Node Name	http://deathnote.vuln/wordpress/index.php/2021/07/19/kira/
Method	GET
Attack	
Evidence	
Other Info	
URL	http://deathnote.vuln/wordpress/index.php/category/uncategorized/
Node Name	http://deathnote.vuln/wordpress/index.php/category/uncategorized/
Method	GET
Attack	
Evidence	
Other Info	
URL	http://deathnote.vuln/wordpress/index.php/hint/
Node Name	http://deathnote.vuln/wordpress/index.php/hint/
Method	GET
Attack	
Evidence	
Other Info	
Instances	Systemic
	Modern Web browsers support the Content-Security-Policy and X-Frame-Options HTTP headers. Ensure one of them is set on all web pages returned by your site/app.

Solution	If you expect the page to be framed only by pages on your server (e.g. it's part of a FRAMESET) then you'll want to use SAMEORIGIN, otherwise if you never expect the page to be framed, you should use DENY. Alternatively consider implementing Content Security Policy's "frame-ancestors" directive.
Reference	https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/X-Frame-Options
CWE Id	1021
WASC Id	15
Plugin Id	10020

Low	**In Page Banner Information Leak**
Description	The server returned a version banner string in the response content. Such information leaks may allow attackers to further target specific issues impacting the product and version in use.
URL	http://deathnote.vuln/sitemap.xml
Node Name	http://deathnote.vuln/sitemap.xml
Method	GET
Attack	
Evidence	Apache/2.4.38
Other Info	There is a chance that the highlight in the finding is on a value in the headers, versus the actual matched string in the response body.
Instances	1
Solution	Configure the server to prevent such information leaks. For example: Under Tomcat this is done via the "server" directive and implementation of custom error pages. Under Apache this is done via the "ServerSignature" and "ServerTokens" directives.
Reference	https://owasp.org/www-project-web-security-testing-guide/v41/4-Web_Application_Security_Testing/08-Testing_for_Error_Handling/
CWE Id	497
WASC Id	13
Plugin Id	10009

Low	**Private IP Disclosure**
Description	A private IP (such as 10.x.x.x, 172.x.x.x, 192.168.x.x) or an Amazon EC2 private hostname (for example, ip-10-0-56-78) has been found in the HTTP response body. This information might be helpful for further attacks targeting internal systems.
URL	http://deathnote.vuln/wordpress/index.php/2021/07/19/kira/feed/
Node Name	http://deathnote.vuln/wordpress/index.php/2021/07/19/kira/feed/
Method	GET
Attack	
Evidence	192.168.1.54
Other Info	192.168.1.54
URL	http://deathnote.vuln/wordpress/index.php/author/kira/feed/
Node Name	http://deathnote.vuln/wordpress/index.php/author/kira/feed/

Method	GET
Attack	
Evidence	192.168.1.54
Other Info	192.168.1.54
URL	http://deathnote.vuln/wordpress/index.php/category/uncategorized/feed/
Node Name	http://deathnote.vuln/wordpress/index.php/category/uncategorized/feed/
Method	GET
Attack	
Evidence	192.168.1.54
Other Info	192.168.1.54
URL	http://deathnote.vuln/wordpress/index.php/comments/feed/
Node Name	http://deathnote.vuln/wordpress/index.php/comments/feed/
Method	GET
Attack	
Evidence	192.168.1.54
Other Info	192.168.1.54
URL	http://deathnote.vuln/wordpress/index.php/feed/
Node Name	http://deathnote.vuln/wordpress/index.php/feed/
Method	GET
Attack	
Evidence	192.168.1.54
Other Info	192.168.1.54
URL	http://deathnote.vuln/wordpress/index.php/wp-json/wp/v2/pages/34
Node Name	http://deathnote.vuln/wordpress/index.php/wp-json/wp/v2/pages/34
Method	GET
Attack	
Evidence	192.168.1.54
Other Info	192.168.1.54
URL	http://deathnote.vuln/wordpress/index.php/wp-json/wp/v2/posts/1
Node Name	http://deathnote.vuln/wordpress/index.php/wp-json/wp/v2/posts/1
Method	GET
Attack	
Evidence	192.168.1.54
Other Info	192.168.1.54

Instances	7
Solution	Remove the private IP address from the HTTP response body. For comments, use JSP/ASP /PHP comment instead of HTML/JavaScript comment which can be seen by client browsers.
Reference	https://datatracker.ietf.org/doc/html/rfc1918
CWE Id	497
WASC Id	13
Plugin Id	2

Low	**Server Leaks Version Information via "Server" HTTP Response Header Field**
Description	The web/application server is leaking version information via the "Server" HTTP response header. Access to such information may facilitate attackers identifying other vulnerabilities your web/application server is subject to.
URL	http://deathnote.vuln
Node Name	http://deathnote.vuln
Method	GET
Attack	
Evidence	Apache/2.4.38 (Debian)
Other Info	
URL	http://deathnote.vuln/robots.txt
Node Name	http://deathnote.vuln/robots.txt
Method	GET
Attack	
Evidence	Apache/2.4.38 (Debian)
Other Info	
URL	http://deathnote.vuln/sitemap.xml
Node Name	http://deathnote.vuln/sitemap.xml
Method	GET
Attack	
Evidence	Apache/2.4.38 (Debian)
Other Info	
URL	http://deathnote.vuln/wordpress
Node Name	http://deathnote.vuln/wordpress
Method	GET
Attack	
Evidence	Apache/2.4.38 (Debian)
Other Info	
URL	http://deathnote.vuln/wordpress/
Node Name	http://deathnote.vuln/wordpress/

Method	GET
Attack	
Evidence	Apache/2.4.38 (Debian)
Other Info	
Instances	Systemic
Solution	Ensure that your web server, application server, load balancer, etc. is configured to suppress the "Server" header or provide generic details.
Reference	https://httpd.apache.org/docs/current/mod/core.html#servertokens https://learn.microsoft.com/en-us/previous-versions/msp-n-p/ff648552(v=pandp.10) https://www.troyhunt.com/shhh-dont-let-your-response-headers/
CWE Id	497
WASC Id	13
Plugin Id	10036

Low	**X-Content-Type-Options Header Missing**
Description	The Anti-MIME-Sniffing header X-Content-Type-Options was not set to 'nosniff'. This allows older versions of Internet Explorer and Chrome to perform MIME-sniffing on the response body, potentially causing the response body to be interpreted and displayed as a content type other than the declared content type. Current (early 2014) and legacy versions of Firefox will use the declared content type (if one is set), rather than performing MIME-sniffing.
URL	http://deathnote.vuln
Node Name	http://deathnote.vuln
Method	GET
Attack	
Evidence	
Other Info	This issue still applies to error type pages (401, 403, 500, etc.) as those pages are often still affected by injection issues, in which case there is still concern for browsers sniffing pages away from their actual content type. At "High" threshold this scan rule will not alert on client or server error responses.
URL	http://deathnote.vuln/robots.txt
Node Name	http://deathnote.vuln/robots.txt
Method	GET
Attack	
Evidence	
Other Info	This issue still applies to error type pages (401, 403, 500, etc.) as those pages are often still affected by injection issues, in which case there is still concern for browsers sniffing pages away from their actual content type. At "High" threshold this scan rule will not alert on client or server error responses.
URL	http://deathnote.vuln/wordpress/
Node Name	http://deathnote.vuln/wordpress/
Method	GET
Attack	
Evidence	
Other	This issue still applies to error type pages (401, 403, 500, etc.) as those pages are often still affected by injection issues, in which case there is still concern for browsers sniffing pages

Info	away from their actual content type. At "High" threshold this scan rule will not alert on client or server error responses.
URL	http://deathnote.vuln/wordpress/index.php/2021/07/19/kira/
Node Name	http://deathnote.vuln/wordpress/index.php/2021/07/19/kira/
Method	GET
Attack	
Evidence	
Other Info	This issue still applies to error type pages (401, 403, 500, etc.) as those pages are often still affected by injection issues, in which case there is still concern for browsers sniffing pages away from their actual content type. At "High" threshold this scan rule will not alert on client or server error responses.
URL	http://deathnote.vuln/wordpress/index.php/hint/
Node Name	http://deathnote.vuln/wordpress/index.php/hint/
Method	GET
Attack	
Evidence	
Other Info	This issue still applies to error type pages (401, 403, 500, etc.) as those pages are often still affected by injection issues, in which case there is still concern for browsers sniffing pages away from their actual content type. At "High" threshold this scan rule will not alert on client or server error responses.
Instances	Systemic
Solution	Ensure that the application/web server sets the Content-Type header appropriately, and that it sets the X-Content-Type-Options header to 'nosniff' for all web pages. If possible, ensure that the end user uses a standards-compliant and modern web browser that does not perform MIME-sniffing at all, or that can be directed by the web application /web server to not perform MIME-sniffing.
Reference	https://learn.microsoft.com/en-us/previous-versions/windows/internet-explorer/ie-developer/compatibility/gg622941(v=vs.85) https://owasp.org/www-community/Security_Headers
CWE Id	693
WASC Id	15
Plugin Id	10021

Informational	**Charset Mismatch**
Description	This check identifies responses where the HTTP Content-Type header declares a charset different from the charset defined by the body of the HTML or XML. When there's a charset mismatch between the HTTP header and content body Web browsers can be forced into an undesirable content-sniffing mode to determine the content's correct character set. An attacker could manipulate content on the page to be interpreted in an encoding of their choice. For example, if an attacker can control content at the beginning of the page, they could inject script using UTF-7 encoded text and manipulate some browsers into interpreting that text.
URL	http://deathnote.vuln/wordpress/index.php/wp-json/oembed/1.0/embed?format=xml&url=http%3A%2F%2Fdeathnote.vuln%2Fwordpress%2Findex.php%2F2021%2F07%2F19%2Fkira%2F
Node Name	http://deathnote.vuln/wordpress/index.php/wp-json/oembed/1.0/embed (format,url)
Method	GET
Attack	

Evidence	
Other Info	There was a charset mismatch between the HTTP Header and the XML encoding declaration: [UTF-8] and [null] do not match.
Instances	1
Solution	Force UTF-8 for all text content in both the HTTP header and meta tags in HTML or encoding declarations in XML.
Reference	https://code.google.com/archive/p/browsersec/wikis/Part2.wiki#Character_set_handling_and_detection
CWE Id	436
WASC Id	15
Plugin Id	90011

Informational	**Information Disclosure - Suspicious Comments**
Description	The response appears to contain suspicious comments which may help an attacker.
URL	http://deathnote.vuln/wordpress/wp-content/themes/twentytwentyone/assets/js/primary-navigation.js?ver=1.3
Node Name	http://deathnote.vuln/wordpress/wp-content/themes/twentytwentyone/assets/js/primary-navigation.js (ver)
Method	GET
Attack	
Evidence	from
Other Info	The following pattern was used: \bFROM\b and was detected in likely comment: "/** * Trap keyboard navigation in the menu modal. * Adapted from TwentyTwenty */", see evidence field for the suspicious comment/snippet.
Instances	1
Solution	Remove all comments that return information that may help an attacker and fix any underlying problems they refer to.
Reference	
CWE Id	615
WASC Id	13
Plugin Id	10027

Informational	**User Agent Fuzzer**
Description	Check for differences in response based on fuzzed User Agent (eg. mobile sites, access as a Search Engine Crawler). Compares the response statuscode and the hashcode of the response body with the original response.
URL	http://deathnote.vuln/wordpress/index.php/category/uncategorized/
Node Name	http://deathnote.vuln/wordpress/index.php/category/uncategorized/
Method	GET
Attack	Mozilla/4.0 (compatible; MSIE 8.0; Windows NT 6.1)
Evidence	
Other Info	
URL	http://deathnote.vuln/wordpress/index.php/hint/
Node Name	http://deathnote.vuln/wordpress/index.php/hint/
Method	GET

Attack	Mozilla/4.0 (compatible; MSIE 6.0; Windows NT 5.1)
Evidence	
Other Info	
URL	http://deathnote.vuln/wordpress/index.php/hint/
Node Name	http://deathnote.vuln/wordpress/index.php/hint/
Method	GET
Attack	Mozilla/4.0 (compatible; MSIE 7.0; Windows NT 6.0)
Evidence	
Other Info	
URL	http://deathnote.vuln/wordpress/index.php/hint/
Node Name	http://deathnote.vuln/wordpress/index.php/hint/
Method	GET
Attack	Mozilla/4.0 (compatible; MSIE 8.0; Windows NT 6.1)
Evidence	
Other Info	
URL	http://deathnote.vuln/wordpress/index.php/hint/
Node Name	http://deathnote.vuln/wordpress/index.php/hint/
Method	GET
Attack	Mozilla/5.0 (Windows NT 10.0; Trident/7.0; rv:11.0) like Gecko
Evidence	
Other Info	
Instances	Systemic
Solution	
Reference	https://owasp.org/wstg
CWE Id	
WASC Id	
Plugin Id	10104

Informational	**User Controllable HTML Element Attribute (Potential XSS)**
Description	This check looks at user-supplied input in query string parameters and POST data to identify where certain HTML attribute values might be controlled. This provides hot-spot detection for XSS (cross-site scripting) that will require further review by a security analyst to determine exploitability.
URL	http://deathnote.vuln/wordpress/index.php/2021/07/19/kira/?replytocom=1
Node Name	http://deathnote.vuln/wordpress/index.php/2021/07/19/kira/ (replytocom)
Method	GET
Attack	
Evidence	

Other Info	User-controlled HTML attribute values were found. Try injecting special characters to see if XSS might be possible. The page at the following URL: http://deathnote.vuln/wordpress /index.php/2021/07/19/kira/?replytocom=1 appears to include user input in: a(n) [meta] tag [content] attribute The user input found was: replytocom=1 The user-controlled value was: width=device-width, initial-scale=1
Instances	1
Solution	Validate all input and sanitize output it before writing to any HTML attributes.
Reference	https://cheatsheetseries.owasp.org/cheatsheets/Input_Validation_Cheat_Sheet.html
CWE Id	20
WASC Id	20
Plugin Id	10031